A Web-Based Solution for Federated Learning With LLM-Based Automation

Chamith Mawela, Chaouki Ben Issaid, *Member, IEEE*, and Mehdi Bennis, *Fellow, IEEE*

***Abstract*—Federated learning (FL) offers a promising approach for collaborative machine learning (ML) across distributed devices. However, its adoption is hindered by the complexity of building reliable communication architectures and the need for expertise in both ML and network programming. This article presents a comprehensive solution that simplifies the orchestration of FL tasks while integrating intent-based automation. A user-friendly web application is developed supporting the federated averaging (FedAvg) algorithm, enabling users to configure parameters through an intuitive interface. The backend solution efficiently manages communication between the parameter server and edge nodes. Model compression and scheduling algorithms are implemented to optimize FL performance. Additionally, intent-based automation in FL is explored using a fine-tuned Language Model (LLM) trained on a tailored dataset, enabling users to perform FL tasks through high-level prompts. It is shown that the LLM-based automated solution achieves comparable test accuracy to the standard web-based solution while reducing transferred bytes by up to 64% and CPU time by up to 46% for FL tasks. Furthermore, neural architecture search (NAS) and hyperparameter optimization (HPO) are leveraged using the LLM to enhance performance, resulting in a 10%–20% improvement in test accuracy for the conducted FL tasks.**

***Index Terms*—Distributed computing, federated learning (FL), intent-based automation, large language models (LLMs), web sockets communication.**

I. Introduction

THE RECENT advancements in machine learning (ML) have led to a surge in organizations adopting these technologies in their business solutions, driven by the abundance of vast amounts of data generated by interconnected devices, particularly in the Internet of Things (IoT) [1]. IoT solutions, such as health care [2], smart cities [3], and autonomous vehicles [4], continuously produce sensitive user data essential for building intelligent models. Traditional ML methods within IoT networks involve a centralized model training approach where data generated by edge nodes within the network must be shared with a centralized server. However, this approach poses a threat to user privacy, as raw client data must be shared with an external party. For example in a healthcare IoT setting, sensitive patient data collected through an IoT device must be shared with a centralized repository for model training. Federated learning (FL) [5] has emerged as a novel paradigm that enables decentralized model training without sharing raw data. FL addresses this issue by enabling collaborative learning where only model weights are communicated among edge nodes and a parameter server (PS), minimizing the risk of private client data leakage. Since its inception by Google [5], FL has been adopted across various domains, such as global COVID diagnosis [6], vehicular networks [7], and wireless communications [8], [9], [10]. Recent studies have explored the application of FL in IoT environments to address data privacy and communication efficiency. For instance, Nguyen et al. [11] demonstrated the effectiveness of FL in healthcare IoT, while Elbir et al. [12] investigated FL as an enabler for smart transportation systems, such as autonomous driving and route planning. Additionally, FL has shown promising results in the smart industry [13], where IoT sensors across multiple factories contribute to predictive maintenance models without sharing sensitive production data. In the context of smart homes, Aïvodji et al. [14] have proposed a novel architecture combining FL with secure data aggregation to enhance security and privacy.

Despite its potential, the adoption of FL is hindered by the complexity of building reliable communication architectures and the need for ML and network programming expertise. In fact, researchers and stakeholders participating in FL implementations must rely on programming expertise and develop seamless communication architectures between edge nodes and the PS. The FL community primarily relies on frameworks, such as Flower [15], PySyft [16], and TensorFlow Federated [17], which require both programming and FL domain expertise. Flower and PySyft follow a similar design architecture where the PS and the client services are programmed separately and deployed into respective devices. Flower uses the remote procedure call (gRPC) protocol for communication between clients and PS while PySyft uses WebSockets. TensorFlow Federated on the other hand is developed only for standalone simulation of FL setting on a single device.

In frameworks, such as Flower and PySyft, the model architecture and hyperparameters for the FL task are typically hardcoded within the PS or client programs. This setup presents a significant challenge during iterative experimentation, as modifications to the model or its parameters require altering the code on either the PS or client devices. Consequently, only individuals with access to these devices

Received 19 August 2024; revised 27 October 2024, 10 December 2024, and 24 January 2025; accepted 6 February 2025. Date of publication 17 February 2025; date of current version 9 June 2025. The work was supported by the European Union through the Project 6G-INTENSE G.A under Grant 101139266. *(Corresponding author: Chamith Mawela.)*

The authors are with the Centre of Wireless Communications, University of Oulu, 90570 Oulu, Finland (e-mail: chamith.mawelamudiyanselage@oulu.fi; chaouki.benissaid@oulu.fi; mehdi.bennis@oulu.fi).

Digital Object Identifier 10.1109/JIOT.2025.3542897

© 2025 The Authors. This work is licensed under a Creative Commons Attribution 4.0 License.
For more information, see https://creativecommons.org/licenses/by/4.0/

TABLE I
COMPARISON OF OUR PROPOSED FRAMEWORK WITH EXISTING SOLUTIONS

Feature	Our Solution	Flower	PySyft	TensorFlow Federated
Web-based solution (No programming required)	✓	×	×	×
Built-in model compression	✓	×	×	×
Built-in scheduling algorithms	✓	×	×	×
Distributed implementation	✓	✓	✓	×
FL without direct device access	✓	×	×	×
Communication protocol	WebSockets	gRPC	WebSockets	N/A
Intent-based automation of FL using LLM	✓	×	×	×
Automated NAS and HPO for FL	✓	×	×	×

can conduct iterative experiments on the FL process by adjusting the parameters. This motivates us to develop a web-based solution where the clients and PS serve as application programmable interface (API) endpoints. This eliminates the requirement for direct access to any device to change parameters since it can be done using the API endpoint. The proposed solution eliminates the need for programming to perform FL tasks by providing functionality through a web application. Another major drawback of the frameworks mentioned above is the lack of inherent support for model compression schemes. When the size of the model used for the FL process becomes bigger, the possibility of communication bottlenecks occurring could become higher in limited bandwidth communication settings. Hence to alleviate the potential bottlenecks in the FL communication process, our proposed solution will provide support for model compression schemes, such as quantization [18], [19] and sparsification [20]. Table I summarizes the main differences between the proposed framework and existing solutions.

More recently, due to the success of large language models (LLMs) and their ability to fine-tune downstream tasks, a wide use of LLMs for automating various tasks can be observed. LLM-integrated tasks span from implementing co-pilots to fully automated solutions. Most of the solutions follow the same structure where an intent or instruction is given by a third party to the LLM where the intent holds necessary information for the LLM to proceed further to produce the intended output. Mostly the instruction is generated by a human or in some cases by the program itself. For instance, LLMs were used to solve math problems [21], automate real-world API consumption [22], generate Verilog and Golang code [23], [24], and automate IT infrastructure management using few-shot learning [25] or repair Ansible scripts for edge cloud infrastructure management [26]. The utilization of LLMs for automating IoT system configurations has been explored in recent works where, Cui et al. [27] proposed LLMind, an LLM-based framework enabling IoT devices to execute complex tasks via natural language by integrating domain-specific AI modules. Similarly, Zong et al. [28] demonstrated LLMs' efficacy in IoT networks, showcasing high accuracy in DDoS detection, macro-programming, and sensor data analysis. Moreover, An et al. [29] introduced IoT-LLM, a framework that enhances LLMs' physical world understanding through IoT sensor data integration and chain-of-thought reasoning, achieving significant improvements in IoT task performance.

In FL tasks, the performance heavily depends on the architecture of the global model and the hyperparameters employed during training. In traditional ML contexts, neural architecture search (NAS) [30] and hyperparameter optimization (HPO) [31] are commonly utilized to identify the optimal model architecture and hyperparameters tailored to a specific dataset. Hence, in our work, utilization of NAS and HPO for performance improvement of the LLM-based automated solution is investigated. Usually, NAS and HPO involve creating a search space for model architectures and hyperparameters, respectively, and carrying out an exhaustive search on the created search space to find the optimal configuration. Due to the dynamic nature of the automated solution and the capability of the LLM to generate model architectures, an LLM is employed to define the search space for NAS.

To the best of our knowledge, there is no existing research in the literature that addresses the automation of FL through user prompts. This study aims to fill this gap by exploring the potential of leveraging user prompts to automate the FL process, making it accessible for users with less FL domain expertise. The main challenge in implementing FL tasks using multiple hardware entities as edge nodes and a PS is the need to build a reliable communication architecture from scratch. Our goal is to bridge the gap between conceptualizing and implementing an FL solution. The limitation posed by other frameworks will be addressed by making the FL process accessible to parties without direct access to the devices running PS and clients. Additionally, a solution is proposed to automate the FL process based on user prompts, which is expected to make FL tasks more accessible to users with less domain knowledge. Accordingly, a web-based solution is introduced with an efficient communication architecture developed using WebSockets to facilitate the orchestration of FL tasks. Moreover, an intent-based platform is integrated into

the developed solution, utilizing a fine-tuned LLM to perform FL operations based on user prompts. Our contributions are as follows.

1) A standard web application is developed to support the federated averaging (FedAvg) algorithm [5] for performing FL tasks. A user-friendly web interface is developed so users can easily select the required parameters for the FL tasks. Furthermore, an efficient communication architecture based on WebSockets is developed as the back-end solution to handle communication between the PS and the edge nodes.
2) The implementation of secondary functionalities in FL is explored, including model compression to optimize the transmission of model weights and scheduling algorithms to manage client participation in the FL task.
3) The intent-based automation of the developed solution is investigated, where FL tasks are performed according to a provided high-level prompt. To achieve this, an open-source LLM is fine-tuned on a tailored dataset and integrated into the solution to automate the FL process based on user prompts.
4) To further enhance the performance of the automated solution, NAS and HPO are implemented. Specifically, the use of LLM is investigated to generate the search space for NAS, and a selective halving-based approach is developed for the HPO process to identify the optimal model architecture and hyperparameters for a specific FL task.

The remainder of this article is organized as follows. FL and related secondary functionalities are detailed in Section II. Section III gives a brief introduction to LLM and their fine-tuning process. The system design and the methodology of the proposed solution are discussed in Section IV. An evaluation of the experimental results for the provided solution is discussed in Section V. Finally, this article concludes with final remarks in Section VI.

II. Federated Learning

FL is a paradigm of ML where models are trained collaboratively using multiple edge nodes (clients), and a PS. Unlike traditional centralized ML training, data is not communicated to a central server. Instead, model optimization occurs locally on the clients, preserving data privacy. The objective of FL is to minimize the weighted average of the local objective functions of each client as follows:

$$\min_{w\in\mathbb{R}^d} f(w) = \sum_{k=1}^{K} \frac{n_k}{n} F_k(w) \tag{1}$$

where $F_k(w)$ is the local objective function for the client k. It is assumed here that the total dataset is partitioned among K number of clients where $n_k = |\mathcal{P}_k|$ and $\mathcal{P}_k$ is the set of data points assigned for the kth client. With these objectives in mind, FedAvg has been proposed in [5]. Considered one of the most widely used and influential algorithms in the realm of FL, FedAvg has inspired numerous extensions and improvements. For the remainder of this article, the implementation is limited to using FedAvg as the FL algorithm.

One of the major challenges in FL is scalability. As the model size and the number of clients grow, encountering a communication bottleneck in FL training may become inevitable. To alleviate the communication bottleneck related to the model size, model compression schemes have been adopted in the FL literature. Next, some common techniques for reducing the size of models are reviewed.

A. Quantization

Quantization is a technique for making models compact by reducing the precision of tensors. Usually, the 32-bit precision of *float32* is reduced to a lower precision, e.g., an 8-bit precision of *int8*. A mapping between the floating point x and its quantization x_q is defined as

$$\mathcal{Q} : x \in [\alpha, \beta] \rightarrow x_q \in \left[\alpha_q, \beta_q\right]. \tag{2}$$

The quantization function $\mathcal{Q}(\cdot)$ [18] is formulated using *affine mapping* between floating point value and its quantized value, and using the *clip* function as follows:

$$\mathcal{Q}(x, S, Z) = \text{clip}\left(\text{round}\left(\frac{x}{S} + Z\right), \alpha_q, \beta_q\right) \tag{3}$$

where the clipping function is defined as

$$\text{clip}(x, \alpha, \beta) = \begin{cases} \alpha, & \text{if } x < \alpha \\ x, & \text{if } \alpha \le x \le \beta \\ \beta, & \text{if } x > \beta. \end{cases} \tag{4}$$

Here, S is an arbitrary positive real number, and Z maps the 0 in floating value to the corresponding value in quantized data type. S and Z represent the scale and zero point quantization parameters, respectively. The recovery of the floating point value through de-quantization can be done using the following:

$$f_d\left(x_q, S, Z\right) = S\left(x_q - Z\right). \tag{5}$$

When applying quantization in the FL setting, locally trained models are quantized and communicated to the PS, where de-quantization is done to recover the original model weights before doing the model aggregation.

B. Sparsification

Sparsification [20] is an alternative approach to model compression, which involves selectively retaining only a subset of the gradients through a sparse selection process. The most widely used sparsification methods in FL are Top_k and $Rand_k$ where given a parameter $1 \le k \le d$ for a gradient vector $x \in \mathbb{R}^d$, only a k subset of gradients selected will be communicated to the PS.

1) Top_k: The top k percent of gradients with the highest absolute values are selected while other values are set to zero. $Top_k : \mathbb{R}^d \rightarrow \mathbb{R}^d$ is formulated as follows:

$$\left[Top_k(x)\right]_j := \begin{cases} [x]_{\alpha(j)}, & \text{if } j \le k \\ 0, & \text{otherwise.} \end{cases} \tag{6}$$

Here, α is a permutation of d such that $|[x]_{\alpha(j)}| \ge |[x]_{\alpha(j+1)}|$ for $j \in [1, d-1]$.

2) $Rand_k$: The k percent of gradients are selected based on a random permutation of the list of gradients. $Rand_k$:$\mathbb{R}^d \times \Omega_k \rightarrow \mathbb{R}^d$ is formulated as follows:

$$[Rand_k(x, \omega)]_j := \begin{cases} [x]_j, & \text{if } j \in \omega \\ 0, & \text{otherwise.} \end{cases} \tag{7}$$

Here, ω is a subset chosen from Ω_k where, $\Omega_k = \binom{[d]}{k}$ represents set of all the k-element subsets of $[d]$.

In the FL context, clients communicate only a sparse selection of k percent of gradients and their corresponding values to the PS, thereby improving communication efficiency.

III. Large Language Models

LLMs have revolutionized the AI community. They can understand and generate human-like text across various domains. Models like GPT [32], [33] have made significant progress in natural language processing (NLP) tasks, such as machine translation, question-answering, programming, reasoning, and decision-making. This spectacular performance of LLMs can be attributed to the large number of model parameters trained on a huge text corpus typically extracted from the web.

The success of LLMs has been further accelerated by large corporations open-sourcing their pretrained language models for use by the general public. *Llama* [34] and *Llama2* [35] which spans models with 7 to 70 B parameters released by *Meta AI*, and *Mistral 7B* [36], *Mixtral 8x7B* [37] by *Mistral.ai* with parameters ranging from 7 to 13 B are some of the widely used open-source LLMs in the community.

A. Transformer Architecture

Every LLM mentioned above occupies some variation of the transformer architecture originally introduced in the article [38]. As depicted in Fig. 1, the transformer architecture is made up of two blocks: 1) an encoder (shown on the left) and 2) a decoder (shown on the right). The transformer architecture is a powerful tool for NLP tasks. Both the encoder and decoder are composed of multiple layers, allowing for the processing and generation of complex language representations. The input text is first tokenized and converted into embedding vectors. Positional encoding is then added to provide information about the position of each token in the sequence, using the following:

$$PE_{(pos,2i)} = \sin\left(\frac{pos}{10000^{\frac{2i}{d_{model}}}}\right) \tag{8}$$

$$PE_{(pos,2i+1)} = \cos\left(\frac{pos}{10000^{\frac{2i}{d_{model}}}}\right). \tag{9}$$

Here, pos represents the absolute position of the token in a sequence of tokens, while $i \in [0, d_{model}/2]$ represents the corresponding dimension related to the embedding vector.

The embedded vectors are passed through the multihead attention layer, which allows the model to focus on different parts of the input simultaneously. The attention mechanism is computed using

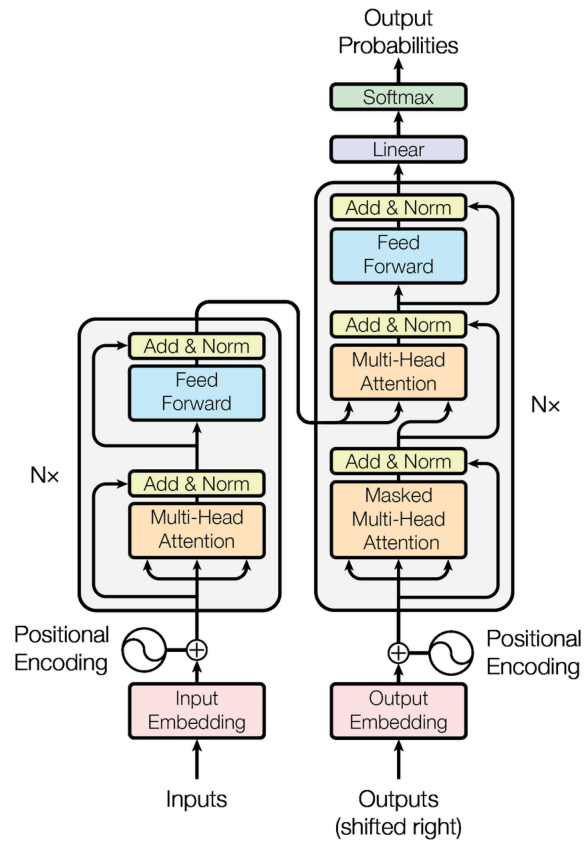

Fig. 1. Transformer architecture: encoder on the left block and decoder on the right block.[1]

$$Attention(Q, K, V) = softmax\left(\frac{QK^T}{\sqrt{d_k}}\right)V \tag{10}$$

where Q is the query vector, K is the key vector with dimensions d_k, and V is the value vector with dimension d_v acting as the input to the attention function.

The output of the attention layer is then passed through a feed-forward layer and residual connections are used to prevent vanishing gradients. The operation of each sublayer can be summarized as

$$output = LayerNorm(x + Function(x)) \tag{11}$$

where, x and output represent the input and output of any sublayer, respectively, while Function can be either multihead attention or feed-forward network function.

The decoder block is similar to the encoder but includes a masked multihead attention layer to ensure that the model only attends to previous tokens when generating new ones. The output of the decoder is passed through a linear layer and softmax function to predict the probabilities of the next token in the sequence. The transformer architecture can be used in various configurations, including encoder-only, decoder-only, and encoder–decoder models, depending on the specific task at hand.

[1]The transformer model architecture originally appeared in [38] with granted permission to reproduce figures for scholarly works.

B. Parameter Efficient Fine-Tuning of LLMs

Initially, LLMs are trained on large text corpus which possesses general knowledge. Hence, when adopting an LLM for downstream tasks, fine-tuning the model with a tailored dataset specific to the task at hand may be necessary. Fine-tuning dataset usually comes in the form $\{(x_i, y_i)\}_{i=1}^{n}$ where x_i is the input to the model and y_i is the target output from the model. Both x_i and y_i are in the form of a sequence of words or tokens. However, fine-tuning an LLM fully with all the parameters included is a huge computationally expensive task requiring large Gigabytes (GB) of virtual random access memory (VRAM). Therefore, parameter-efficient fine-tuning (PEFT) methods have been explored in the literature.

LoRA [39] is a widely used method for performing efficient fine-tuning on LLMs allegedly reducing trainable parameters by 10 000 folds while lowering the graphical processing unit (GPU) memory requirement by three times compared to fully fine-tuning a GPT-3 175 B parameter model with Adam optimizer. LoRA reduces trainable parameters by decomposing the weight matrices using lower-rank matrices. Consider a pretrained weight matrix $W \in \mathbb{R}^{d\times k}$. In LoRA, the gradient update is represented as follows:

$$\Delta W = BA \tag{12}$$

where $A \in \mathbb{R}^{r\times k}$ and $B \in \mathbb{R}^{d\times r}$, with rank $r << \min(d, k)$. The forward pass is calculated as

$$h = Wx + \Delta Wx = Wx + BAx \tag{13}$$

where x is the input embedding or intermediate hidden features. Once the fine-tuning is done, inference for the downstream tasks is done by merging the weights as follows:

$$W = W + BA. \tag{14}$$

By this means, adapting LoRA for multiple downstream tasks is possible since it is possible to train different LoRA adapters and plug them into the original model as above.

QLoRA [40] is a variant of LoRA to enhance further the efficiency of fine-tuning an LLM. This is achieved by utilizing a quantized version of the transformer model, double quantization, and using paged optimizers. The transformer model is quantized to 4-bit precision using the 4-bit Normal Float (NF4) data type. Hence, QLoRA reduces the GPU memory requirement for fine-tuning a 65B model to less than 48 GB, compared to more than 780 GB in full fine-tuning.

IV. System Design and Methodology

As discussed in previous sections, our goal is to implement a web application to make FL accessible for everyone without dealing with the intricacies of network programming to implement a communication architecture. To that end, a new web application is developed with an inbuilt communication architecture to handle FL tasks, thereby eliminating the need for programming an FL solution from scratch. Our solution is mainly divided into three phases:

1) *Phase 1:* A standard web solution is developed to facilitate FL tasks.

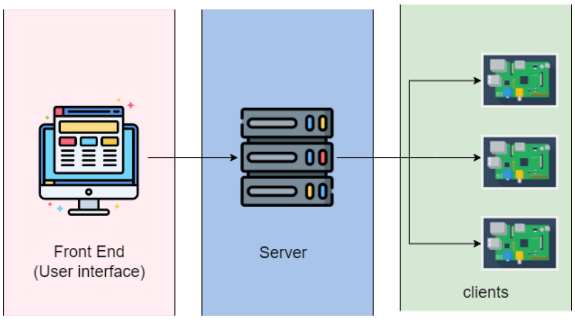

Fig. 2. Overview of the web application FL solution.

2) *Phase 2:* An LLM is fine-tuned and integrated to further automate the developed solution.
3) *Phase 3:* NAS and HPO using LLM are explored to further improve the results of the automated solution.

A. Standard Web Application

Our proposed solution comprises three key elements, as illustrated in Fig. 2. The front-end user interface[2] is designed to enable end users to input essential parameters required for executing an FL task. The server (here onward known as FL server) functions as the central coordinator, responsible for tasks, such as aggregating parameters, scheduling clients, and managing database services. Additionally, individual clients run separate code locally for training, with each client capable of operating on various hardware setups, provided they possess sufficient computational resources for local training and a communication interface for interacting with the server.

The front-end web user interface is developed as a React [41] application, where Redux [42] is used for managing the state of the front-end data. A web form is used in the front end to collect the necessary information for carrying out an FL task. Comprehensive information about the most important fields in the web form to be filled by the end user is explained in the Appendix. The user has to submit the required task information via the web form, after which the data is converted to javascript object notation (JSON) format and transmitted to the FL server to initiate the FL task. An example of the content of a JSON can be found in the Appendix.

WebSockets serve as the communication protocol for all interactions between the front-end and FL server and between the FL server and the clients. WebSockets is a full duplex communication protocol built on top of the transmission control protocol (TCP), which facilitates real-time data transfer between two communicating devices. Furthermore, WebSockets support secure, encrypted connections which is ideal for communication in FL solutions as the security of data is not compromised. In the Appendix, the communication flow of WebSockets is explained, along with the rationale behind choosing WebSockets as the communication protocol.

[2]The front-end and back-end codes are publicly available at https://github.com/ICONgroupCWC/FedLFrontEnd and https://github.com/ICONgroupCWC/FedLBE, respectively.

B. FedAvg Implementation

Building on the foundational FL principles outlined in Section II, the following modifications are made to the original FedAvg algorithm to incorporate model compression and client scheduling.

1) Model compression schemes are implemented for efficient communication of model weights between the client and the FL server. Users can select a compression scheme to compress the locally trained model before transferring the weights back to the PS. The options for the compression scheme are quantization, Top-k or Rand-k, as introduced in Section II.
2) Several scheduling mechanisms are implemented to select clients for each round of communication. Users can choose from the following options.
   a) *Random:* PS selects clients randomly based on a given client fraction C.
   b) *Round Robin:* PS selects clients in a round-robin fashion.
   c) *Latency-Proportional:* PS selects clients with the lowest latency averaged over k communication rounds.
   d) *Full:* PS selects all clients during each communication round.

The FedAvg algorithm has been extended with these modifications and integrated with the developed web application to offer a configurable and high-performance FL experience through the provided web interface. When no model compression scheme is applied and the client selection follows a random mechanism, the system replicates the behavior of the original FedAvg algorithm. In configurations where model compression is enabled, the compression of model weights occurs on the client side before transmission, while decompression is performed on the server side upon receipt.

As illustrated in Fig. 3, the implementation of the modified FedAvg algorithm leverages Web-Sockets for communication, with eight high-level stages. The process begins with the user submitting an FL request through a web form, establishing a WebSocket connection between the front-end web application and the FL server. Then, the server communicates the request to selected clients based on the client fraction and scheduling mechanism, sending the global model and training parameters. Each client trains the model locally and sends the trained weights back to the server asynchronously. The server aggregates the local models to update the global model, calculates test results, and saves the data in a database. Finally, the current communication round's training and test metrics are sent to the front-end via the existing WebSocket connection, updating the web application dashboard in real time with user-selected plots. This process is repeated iteratively until all requested communication rounds are completed, maintaining the WebSocket connection between the front-end and FL server throughout the training.

C. LLM Finetuning

The downstream task in our proposed solution is carrying out an FL task based on the user prompt provided. Hence, the output of the LLM must be compatible with the developed FL server. In our previous case with the standard web application, the payload in the request from the user contained a JSON, which carries information required for the FL task, such as model parameters, optimizer parameters, etc. Hence, it is natural to use the same structure of the JSON as output from the LLM to make the new solution compatible with the previously developed solution.

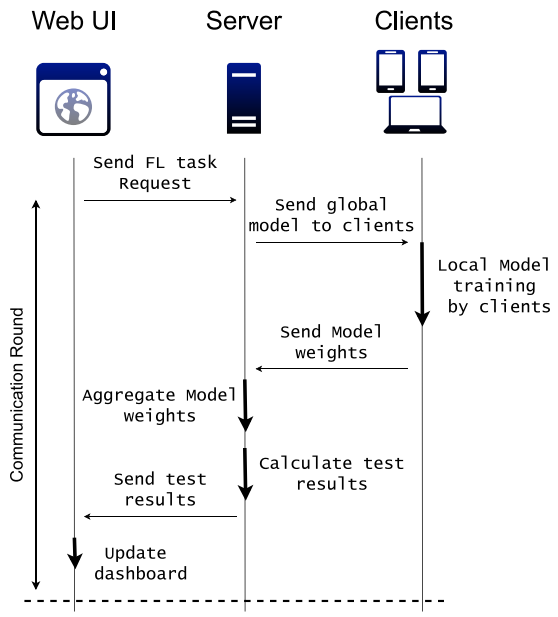

Fig. 3. Communication process between Front-end, server, and clients.

While LLMs can handle many NLP tasks using zero-shot or few-shot learning [43], the proposed automated FL system presents unique requirements that necessitate fine-tuning. Recent works, e.g., [22], [44], demonstrate that fine-tuned models achieve higher accuracy and better generalizability compared to prompt engineering alone. The downstream task requires consistent generation of precisely structured JSON outputs that must be exactly compatible with the FL server's expected format where even minor formatting errors or deviations could cause the FL process to fail. Additionally, the system needs to reliably parse a wide variety of natural language descriptions into the correct parameter values while handling edge cases and variations in how users might express their FL requirements. In such scenarios, the effect of hallucination [45] might lead to catastrophic failures in the FL process by generating syntactically valid but semantically incorrect parameter configurations. Hence in the proposed work, supervised fine-tuning of the LLM is performed ensuring reliable and consistent configuration outputs.

To fine-tune the LLM, a dataset is created with <INTENT, TARGET> pairs, where INTENT represents the user prompt provided, and TARGET is the intended JSON output generated by the LLM for the given intent. For example, the user intent can be as follows:

> **Prompt:** *Create a FL task with MNIST dataset. Run for a total of 20 communication rounds and*

TABLE II
DEFAULT VALUES TO BE USED FOR PARAMETERS WHEN UNSPECIFIED IN THE PROMPT

Key	Value	Definition
minibatch	16	Size of the mini-batch used during training
algorithm	classification	Type of algorithm used for the FL task
epoch	5	Number of epochs for which the local model is trained
lr	0.0001 (Adam) 0.001 (SGD) 0.004 (AdaGrad) 0.0004 (RMSProp)	Learning rate used for optimizing the model
scheduler	full[3]	Scheduling mechanism used to select clients for each communication round
clientFraction	1	Fraction of clients selected for each communication round
minibatchtest	32	Size of the mini-batch used during testing
comRound	10	Number of communication rounds
optimizer	Adam	Optimizer used for training the model
loss	CrossEntropyLoss	Loss function used for training the model
Compress	No	Compression technique used for model weights

12 local epochs. Use SGD as the optimizer with a learning rate of 0.004. Use a mini-batch size of 32 for training. The client fraction should be 0.7.

The target JSON should be accustomed according to the provided intent. One caveat here is when the prompt becomes simple, values to be used in the JSON for the unspecified parameters become unclear. To address the challenge of handling diverse user input prompts and enhancing the flexibility of parameter selection, the fine-tuned LLM is leveraged to parse the user's natural language prompt and directly infer a JSON configuration. Given the variability of user input in terms of phrasing and completeness, the system employs a set of hierarchical and conditional inference rules to assign default values to any omitted parameters as specified in Table II. These rules are designed to prioritize certain parameters based on their dependencies and contextual relevance, for example, the choice of optimizer may influence the recommended learning rate. Each parameter has been carefully selected based on empirical evidence and best practices in FL. Some of these default values like learning rate are derived experimentally by running for common datasets used in our work, such as MNIST. When creating the dataset, emphasis is placed on crafting prompts that effectively capture the inference rules required for parsing user input, enabling the LLM to accurately infer the JSON based on the user prompt. This allows users the flexibility to either specify all intended parameters or leave some unspecified, which will then be inferred by the LLM.

One other aspect considered when developing the dataset is possible variation in phrasing the user's intent. For example, when the `clientFraction` needs to be specified in the intent, it can be phrased in several ways as follows.

Prompt: *Create a FL task with MNIST dataset with a client fraction of 0.7.*
Prompt: *Perform a FL task with MNIST dataset involving 70% of the total clients.*
Prompt: *Execute a FL task with MNIST dataset involving 7/10 of the total clients.*
Prompt: *Start a FL task with MNIST dataset by omitting 30% of total clients in each communication round.*

The output JSON of all above prompts should be the same and the dataset is created by including similar variations throughout the dataset for other parameters as well. Apart from that, several other actions are taken to induce variation to the dataset, such as paraphrasing, differing complexity of the sentence structure, and differing the order of parameters provided in the intent.

To get good results when fine-tuning an LLM, structuring the prompt is very important. Apart from the user's intent and the target's response, a system instruction is added to the input prompt. The system instruction can provide contextual guidance to the model, informing it about the task at hand or the desired behavior. This helps the model understand the specific requirements of the fine-tuning task and adjust its predictions accordingly. In our case, the System Prompt is used to communicate the role of the LLM succinctly as shown in the Appendix. A detailed description of each parameter of the output JSON is given by the System Prompt for helping to predict the output accurately. The system prompt remains static across all data points, as fine-tuning is performed for a single downstream task, and the intended output structure is consistent for all user intents. Finally, each data point is customized using the System Prompt, Intent, and Target before feeding into the fine-tuning pipeline.

D. LLM Integration to the Web Application

Once an open-source LLM model is fine-tuned for our downstream task, the model is integrated into the existing solution. An overview of the extended system design is depicted in Fig. 5. A separate server is implemented[3] to host the LLM model, where a representational state transfer (REST) API endpoint is created using Python Flask to consume the LLM service. The fine-tuned LLM model is converted to the GGUF format before being hosted in the LLM Server. GGUF is a binary file format used for storing LLM models for inference, enabling faster loading and serving of models. A Python binding [46] for llama.cpp [47] is used for serving the model inside the provided REST endpoint.

One challenge for the integrated solution is providing the global model architecture for carrying out the FL task. In the standard web application solution, there is the option of submitting a Pytorch model architecture file. However,

[3]The code for the LLM server is available at https://github.com/ICONgroupCWC/LLMServer.

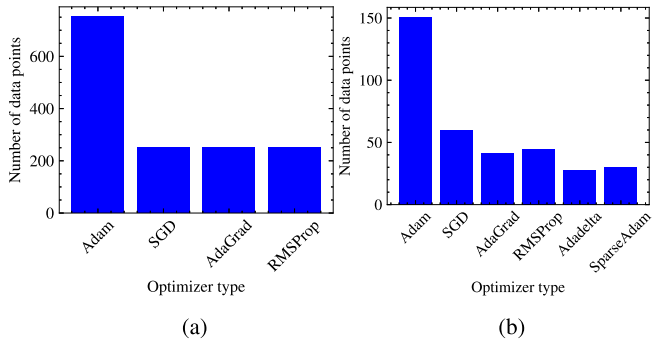

Fig. 4. Optimizer variations across the train and test datasets for fine-tuning the LLM. (a) Training dataset (b) Test dataset.

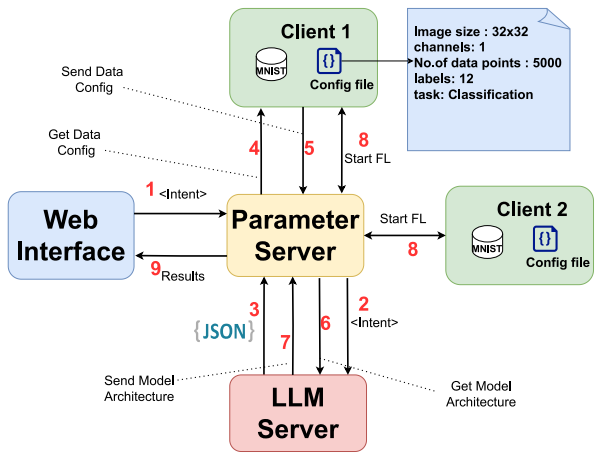

Fig. 5. System design for the LLM integrated FL solution.

given that input for the LLM-integrated solution is in natural language, it is not feasible to provide a model architecture as describing the model architecture in natural language is a complex and tedious task. To overcome this, leveraging the reasoning capabilities of LLMs outlined in Section III, ChatGPT is utilized to propose a model architecture for a given task. Experimental results verify that, given the input and output dimensions of the network and the number of data points, ChatGPT can provide an appropriate model architecture accordingly.

Since data is a commodity of the clients in the FL process, the responsibility of providing the details of the dataset is given to the clients. A configuration file along with the dataset should be kept in the data folder, which holds information, such as the dimension, the number of data points, the task, and the number of labels as depicted in Fig. 5. Using this provided configuration file a new prompt is created to query ChatGPT for a model architecture as follows.

> **User_Prompt:** *Create a model architecture for the following task. The task is a Classification task with 12 labels. The input data is a tensor of shape (1,1,32,32) and has 5000 datapoints altogether. Considering the above information, create a neural network architecture that could achieve good accuracy.*

The above user prompt is always kept static, with placeholders for underlined values where the underlined values are overridden using the configuration file provided in the client data folder. Similar to the previous case, a system prompt with task details is used to support this prompt before the request is sent to the ChatGPT API. Accordingly a separate REST endpoint is implemented in the LLM server to query a model architecture for a given task.

As illustrated in Fig. 5, the communication process of the integrated solution occurs in nine high-level stages. Initially, the user sends the intent for the FL task to the PS. Next, the PS forwards the intent to the LLM server endpoint to get the configuration of the FL task, where the LLM server predicts the configuration using the fine-tuned LLM and returns the configuration file to the PS as the next step. Then, the PS requests a randomly selected client for the configuration file which contains the dataset's properties. Next, the returned configuration file is forwarded to the LLM server, where the LLM server overrides the aforementioned user prompt and queries the ChatGPT API to provide a model architecture suitable for the task. Once the LLM server receives the model architecture, it is returned to the PS by the LLM server. Finally, the FL process is carried out as usual using the received model architecture as the global model and intermediate results are communicated with the front-end.

The key advantage of FL is its inherent privacy-preserving nature by design, which allows model training without direct access to users' sensitive data. In the provided approach, no sensitive information is communicated between clients and the PS, and only a data configuration file is communicated from the clients, which does not hold any sensitive information. Hence integration of LLM does not affect the privacy-preserving nature of the FL.

E. Neural Architecture Search and Hyperparameter Optimization

As discussed in the previous section, it is observed that ChatGPT has the ability to suggest a neural network architecture based on the nature of the task and the dataset. However, one caveat in the proposed methodology is achieving good test accuracy is uncertain since the task is carried out using an arbitrary model suggested by the ChatGPT. Hence, this section focuses on improving test accuracy by leveraging ChatGPT to generate the search space for NAS. Additionally, HPO is performed on the suggested models in the search space to identify the best hyperparameters suited for the respective models. Both NAS and HPO are performed in a distributed manner before the FL task starts. The procedure for performing NAS and HPO is outlined in Algorithm 1 and illustrated in Fig. 6.

According to the provided algorithm, NAS is performed for X number of search rounds as provided in the input. ChatGPT is utilized to initialize the search space at each round, where the *CreateSearchSpace* function generates m models, with m representing the total number of clients participating in the training during the current search round. Two prompts are used in generating model architectures where

Algorithm 1 Algorithm for NAS and HPO for Automated Solution

Inputs: Number of search rounds X, number of HPO rounds H, number of explorer epochs E, total number of clients K, client fraction C, Hyperparameter types T, number of hyperparameters Y, data configurations d, initial batch size B
Output: Final best model n, Best hyperparameters o
for each round $t = 1, 2, ..., X$ **do**
  $m \leftarrow \max(C \cdot K, 1)$
  $N_t \leftarrow$ **CreateSearchSpace**(m, d)
  $S_t \leftarrow$ (set of m clients selected randomly)
  for each client $k \in S_t$ **in parallel do**
    $n_k \in N_t \leftarrow$ (receives model architecture from Server)
    for each round $j = 1, 2, ..., H$ **do**
      $O \leftarrow$ **CreateHPOSpace**(T, Y)
      $o_j, p_j \leftarrow$ **PerformHPO**(n_k, O, E, B)
    end for
    $o_k, p_k \leftarrow$ (where $k = \arg\max p_j$)
    sends (n_k, o_k, p_k) to server
  end for
  $\{(n_k, o_k, p_k) \mid k = 1, 2, 3, \ldots, m\}$ (receives from clients)
  $(n_t, o_t, p_t) \leftarrow$ (where $t = \arg\max_{1 \le k \le m} p_k$)
end for
$(n_f, o_f, p_f) \leftarrow$ (where $f = \arg\max_{1 \le t \le X} p_t$)
$n, o \leftarrow (n_f, o_f)$
PerformHPO(n, O, E, B)
$p_b = 0$ // Stores best performance
$o_b = \{\}$ // Stores best hyperparameter combination
$current_round = 0$
while $O.length > 1$ **do**
  if $current_round \neq 0$ **then**
    $O \leftarrow$ (Select the top performing half of hyperparameters)
    $B \leftarrow \min(B \cdot 2, traindata.size)$
  end if
  $\beta \leftarrow$ (select B datapoints randomly from training data)
  for each combination $o \in O$ **do**
    for each epoch $e = 1, 2, 3 \ldots, E$ **do**
      TrainModel(n, o)
    end for
    $p \leftarrow$ **EvaluateModel**(n, o, β)
    if $p > p_b$ **then**
      $p_b \leftarrow p$
      $o_b \leftarrow o$
    end if
  end for
  $current_round += 1$
end while
return (o_b, p_b)

at the first round *InitialPrompt* is used while at later rounds *IntermediatePrompt* is used. Also, an *ErrorPrompt* is used to handle errors whenever the models produced by the ChatGPT are invalid. The prompts used are as follows: For all prompts, the underlined values are given as input to the function.

InitialPrompt: You are an AI that strictly conforms to responses in Python. You are an assistant in providing CNN architectures as a search space for NAS. I need help designing the optimal CNN architecture for a specific dataset. I plan to start with a variety of models and change the model architecture based on model performance. As the initial step, please provide three CNN architectures with varying width, depth, and layer types. The input data is in the form torch.zeros((1,3,32,32)) and the output is in the form torch.size((1,10)). There are altogether 10 000 data points for training. When suggesting models give importance to the above factors to make the model complex accordingly. After you suggest a design, I will test its performance and provide feedback. Based on the results of previous experiments, we can collaborate to iterate and improve the design. Please avoid suggesting the same design again during this iterative process. Your responses should contain valid Python code only, with no additional comments, explanations, or dialogue. Provide the PyTorch models according to the given prompt. Important!! Output the solution as three different Python code bases. Start each code base with <Code> and end the code base with </Code> brackets. So there must be three brackets. Include PyTorch imports with "import torch," "from torch import nn," and "import torch.nn.functional as F." Do not include model initialization code.

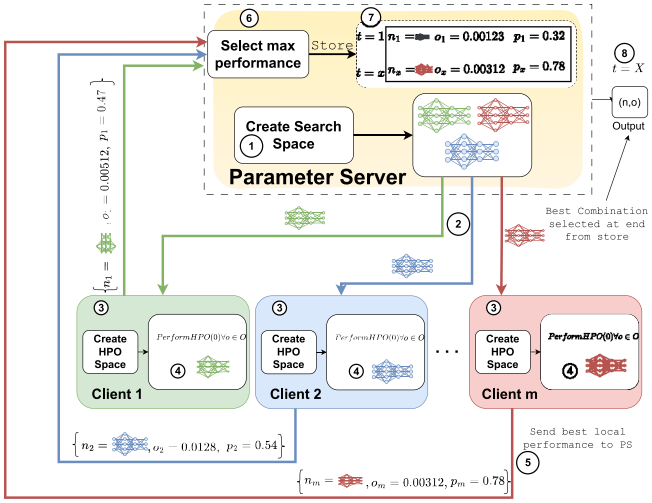

Fig. 6. Federated NAS and HPO process.

Intermediate Prompt: By using provided model model, we achieved an accuracy of 57%. As previously Please recommend three new models that outperform prior architectures based on the above-mentioned experiments on NAS. Take a step toward making the model more complex by adding more layers to the neural network and increasing the number of parameters in the model. The output should be structured the same as in the previous case. As in the previous case, the input is in the form torch.zeros((1,3,32,32)), and the output is in the form torch.size((1,10)). Please make sure that all the dimensions are correct in the network and no error will arise

Error Prompt: The suggested model (model) gives the following error (error message). Please suggest a new model architecture with similar parameter complexity that conforms with the dimensions of the

input and output correctly to avoid the above error. The input is in the form torch.zeros((1,1,32,32)) and output a tensor in the form torch.size((1,10)).

During each search round, the produced m model architectures are sent to the m number of local clients to carry out HPO. For each client, HPO is run for H number of rounds, where at each round, Y hyperparameter configurations are created. In this study, the focus is exclusively on the learning rate as a hyperparameter, as it is considered to have the most significant impact on model performance. Furthermore, focusing on a single hyperparameter significantly reduces the search space. This simplification allows for a more thorough exploration of learning rate values within the given computational constraints. Hence, Y number of learning rates between a provided upper bound of 5×10^{-2} and a lower bound of 10^{-5} will be generated randomly by *CreateHPOSpace* function at each HPO round by each local client.

The created HPO space is passed to the *PerformHPO* function where the HPO operation is carried out using a selective halving manner. At each step after the first step, hyperparameter space is reduced by half size where only the best 50% of the hyperparameters per their performance are retained. Meanwhile, the training set size is also increased by two folds to ensure the retained hyperparameter space is trained on more data points while the HPO progresses. The model architecture received by the local client is trained for E number of epochs, for each retained hyperparameter using the batch of data β in the local client.

Once the training is done, the model is evaluated using a validation dataset to find the best-performing hyperparameter. Here, validation accuracy is used as the performance metric. The best candidate is updated at each round where the *PerformHPO* process is carried out until only one candidate from the HPO space is retained with the highest performance. This procedure is run for H number of HPO operations where the hyperparameter and performance pairs will be recorded. At the end of all the H HPO rounds, the best-performing hyperparameter will be selected from the pool of recorded hyperparameters. Then, the combination of the model, best hyperparameter, and its performance is sent to the server where the server stores these combinations in a list for future comparison. Once all the NAS rounds are completed, the model and hyperparameter combination with the best performance is selected as (n, o) to be used as the global model and learning rate for the specific FL process.

V. Experimental Results

A. Web App Results Under Varying Network Conditions

The performance of the standard web application is first evaluated under varying network conditions with 20 clients. Due to the limited availability of dedicated hardware for each client, all 20 client programs are executed on a single PC with sufficient computing resources. The web app's performance under low bandwidth and high latency conditions is evaluated, where both conditions are simulated. All experiments are conducted using the MNIST dataset with 50 communication rounds, 2 local epochs, and the Adam optimizer with a learning rate of 10^{-4}.

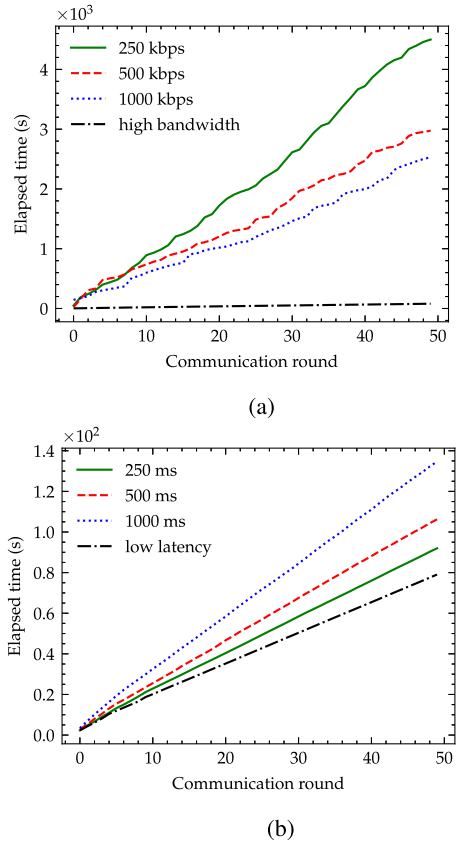

Fig. 7. Web App performance under different network conditions. (a) Varying bandwidth settings. (b) Varying latency settings.

To simulate low bandwidth settings, the bandwidth limit for each client is selected from a Gaussian distribution, and model communication is throttled accordingly. Three different Gaussian distributions are used with different mean values of 250, 500, and 1000 kilobytes per second (Kb/s) to evaluate different low bandwidth settings. Fig. 7(a) illustrates elapsed time against the communication rounds with varying bandwidth settings, where high bandwidth refers to the condition where low bandwidth limitations are not applied for the clients. It is observed that while the FL process proceeds without disruption, the elapsed time becomes high under simulated low bandwidth settings.

Furthermore, to evaluate performance under high-latency settings, similar experiments are conducted where high latency is simulated across clients by selecting the additional latency for each client from a Gaussian distribution with mean values of 250, 500, and 1000 milliseconds (ms). As illustrated in Fig. 7(b) the elapsed time for total communication rounds increases with added latency. Here, low latency represents the situation where high latency conditions are not applied to the clients.

B. LLM Finetuning Results

For the fine-tuning process, a new dataset comprising 1504 data points with <INTENT, TARGET> pairs is created to

fine-tune the LLM.[4] The dataset is created while considering a wider range of possible variations that could occur in the user intent. As illustrated in Fig. 4, several variations of optimizer choices are included in the dataset. As observed in the plots, the number of instances containing default values is higher. This is due to the higher number of data points not specifying the relevant parameter in the intent. For example, it is natural for a user to input only the `clientFraction`, but not the `scheduler`, where in such cases the default value is used.

This dataset is used to fine-tune a Mixtral-7B [36] model. The fine-tuning process is carried out for two epochs through the dataset using the QLoRA method[5] as discussed in Section III. Hence, the model is loaded in 4-bit precision for fine-tuning, where at the end of the training, the LoRA module is merged with the base model by upscaling the trained LoRA to 8-bit precision. An 8-bit precision variation of the AdamW optimizer, AdamW_8bit [48], is used for fine-tuning, with a learning rate of 2×10^{-4}. A single A100 GPU provided by Google Colab is used for fine-tuning where Unsloth [49] library is used for efficient fine-tuning. Unsloth is an open-source library that provides Python API for faster fine-tuning of LLMs.

To evaluate the performance of the fine-tuned Language Model, a new test dataset comprising 355 instances, similar to those in the fine-tuned dataset, is created. The dataset is designed to span from simpler prompts with minimal specified parameters to complex prompts with many parameters included, ensuring a comprehensive evaluation spectrum. Additionally, for categorical parameters, such as optimizers, new values were added to test the generalization capability of the LLM as evident from Fig. 4, where Adadelta and SparseAdam were not used as optimizers in the intents in the fine-tuning stage. Similarly, values for the *dataset* key were also given unseen values with random dataset names to evaluate the generalizability. The evaluation accuracy of the test dataset holds at 99.4%, where only two data points are incorrectly predicted. Some of the prompt-target pairs correctly inferred by the LLM are depicted in Appendix VI-E.

C. Comparative Analysis of Web-FedAvg and LLM-FedAvg Implementations

Several simulations are carried out using both standard and LLM-integrated solutions to evaluate the results. In the remainder of this article, the standard solution is denoted as *Web-FedAvg*, and the LLM-integrated solution is denoted as *LLM-FedAvg*. For all the simulations, three Raspberry Pi modules are used as clients and both the PS and the LLM server are run on a single Laptop with an i7-9750H CPU, 24-GB random access memory (RAM), and a NVIDIA Geoforce 1660Ti GPU with 6-GB dedicated memory, where all the devices are connected to a single local wireless network.

Simulations are run for several cases using MNIST, CIFAR-10, Fashion-MNIST, and SVHN datasets where the dataset is independent and identically distributed (IID) among clients. A summary of scenarios used for simulations is given in Table III. For the cases represented in the table, *LLM-FedAvg* method is carried out by creating a relevant prompt to capture the necessary parameters. The simulation results are tabulated in Table IV for both *Web-FedAvg* and *LLM-FedAvg* methods, where communication overhead and CPU time represent the total number of bytes transferred from the client to the server and the total elapsed time for all the communication rounds, respectively. The number of parameters of the models used by both methods is plotted in Fig. 8, where for *Web-FedAvg*, a handcrafted model is used, while for *LLM-FedAvg*, the model is suggested by ChatGPT given the dataset configuration.

From case 1, it is observed that both cases achieve test accuracy over 98% where *Web-FedAvg* achieves slightly better accuracy. However, communication overhead and CPU time are much less for *LLM-FedAvg*. For case 2, *LLM-FedAvg* seems to achieve around 3% better test accuracy than *Web-FedAvg* while maintaining a lesser communication overhead and CPU time. In case 3, it is observed that both *Web-FedAvg* and *LLM-FedAvg* achieve a similar accuracy of around 89%, with *LLM-FedAvg* achieving these results with reduced communication overhead and CPU time. Finally, in case 4, *LLM-FedAvg* manages to achieve around 3% better accuracy

TABLE III
SUMMARY OF THE DIFFERENT USE CASES USED IN OUR EXPERIMENTS USING MNIST, FASHION-MNIST, CIFAR-10, AND SVHN DATASETS

Cases	Description
Case 1	An FL task using **MNIST** dataset, with a client fraction = 0.7, local minibatch size = 4, number of local epochs = 1, communication rounds = 50, optimizer = Adam, learning rate = 0.0001
Case 2	An FL task using **CIFAR-10** dataset with a client fraction = 0.7, local minibatch size = 8, number of local epochs = 2, communication rounds = 100, and optimizer = Adam, learning rate = 0.001.
Case 3	An FL task using **Fashion-MNIST** dataset, with a client fraction = 0.8, local minibatch size = 8, number of local epochs = 1, communication rounds = 50, optimizer = Adam, learning rate = 0.0001
Case 4	An FL task using **SVHN** dataset with a client fraction = 0.7, local minibatch size = 8, number of local epochs = 1, communication rounds = 50, and optimizer = Adam, learning rate = 0.0001.

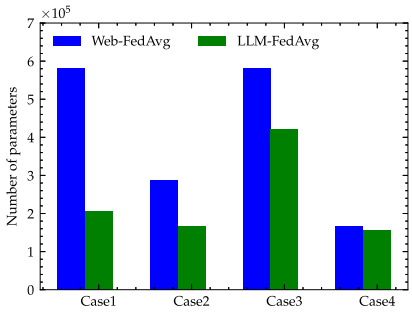

Fig. 8. Number of parameters of the model used for Web-FedAvg and LLM-FedAvg for each case.

[4]The fine-tuned dataset is available at https://huggingface.co/datasets/cmarvolo/autofl.

[5]The fine-tuned LoRA adapter is available at https://huggingface.co/cmarvolo/mistral-7b-fed-auto-lora.

TABLE IV
PERFORMANCE OF WEB-FEDAVG AND LLM-FEDAVG FOR THE DIFFERENT CASES

Cases	Method	Test Accuracy	Communication Overhead (bytes)	CPU (s)
Case 1	Web-FedAvg	98.69%	1.165×10^2	3.761×10^3
	LLM-FedAvg	98.14%	4.150×10	2.004×10^3
Case 2	Web-FedAvg	69.18%	6.700×10	4.290×10^4
	LLM-FedAvg	72.55%	6.276×10	3.880×10^4
Case 3	Web-FedAvg	89.74%	1.165×10^2	1.736×10^3
	LLM-FedAvg	89.69%	8.450×10	1.649×10^3
Case 4	Web-FedAvg	86.30%	3.350×10	1.643×10^4
	LLM-FedAvg	89.87%	3.150×10	1.583×10^4

over *Web-FedAvg* while having lesser communication overhead and elapsed CPU time. In summary, both *Web-FedAvg* and *LLM-FedAvg* seem to achieve similar accuracy for simulations carried out, while sometimes *LLM-FedAvg* achieve slightly better accuracy. However, for all the cases communication overhead and elapsed CPU time for the FL process is lesser in *LLM-FedAvg* than *Web-FedAvg*. The reason is that chatGPT continuously suggests models with lesser parameters as depicted in Fig. 8 for *LLM-FedAvg* for all the cases.

D. Comparative Analysis of LLM-FedAvg and LLM-FedAvgNAS Implementations

Although *LLM-FedAvg* achieves a reasonably good test accuracy, the accuracy can potentially be further improved. Therefore, NAS and HPO are employed, as outlined in Algorithm 1, which is termed as *LLM-FedAvgNAS* here onward. NAS and HPO are performed only on the CIFAR-10 and Fashion-MNIST datasets from the previous experiments. Additionally, CIFAR-100 and Tiny ImageNet datasets are used to evaluate the generalizability of the proposed method. For all the *LLM-FedAvgNAS* search processes, five search rounds (S) with two local HPO rounds (H) are carried out. Only the learning rate is used as the hyperparameter type (T). The total number of epochs (E) is kept as 20 while a client fraction of one is set to use all three clients in the search process. The data distribution among clients is the same as the previous where the data is distributed in an IID setting. The number of hyperparameters (Z) is set as 20, where for each HPO round, 20 random learning rates are created as HPO search space. The initial batch size for all the simulations is kept at 250. Due to the increased compute and memory demand for the search process, the clients, in this case, are run using a single V100 GPU since Raspberry PI compute is insufficient to carry out the search process.

Once the best model and learning rate are obtained using *LLM-FedAvgNAS* search, the FL process is carried out separately using the obtained results for 100 communication rounds for CIFAR-10, Fashion-MNIST and CIFAR-100 datasets and 20 communication rounds for Tiny ImageNet dataset. Only one local epoch with a client fraction of 0.7 is used to utilize only two of three total clients. *LLM-FedAvgNAS* search is carried out three times obtaining three global model and learning rate pairs to plot the mean and standard deviation of the test accuracy of FL communication rounds as illustrated in Fig. 9. Similarly, *LLM-FedAvg* is carried out three times using the same number of communication rounds and client fraction. The prompt used for this only specifies the dataset, communication rounds, local epochs, and client fraction ensuring that default values specified in Table II are used for the rest of the parameters. Similar to the previous case, the mean test accuracy and standard deviation for *LLM-FedAvg* simulations are plotted in Fig. 9.

Observing the results in Fig. 9, it is evident that *LLM-FedAvgNAS* surpasses *LLM-FedAvg* in test accuracy by a significant margin. For the CIFAR-10 dataset, *LLM-FedAvgNAS* manages to achieve a test accuracy of around 76% while for Fashion-MNIST an optimal accuracy of over 98% is achieved. It is observed that for the CIFAR-100 dataset, *LLM-FedAvgNAS* achieves an accuracy of around 42% toward the end of the communication rounds, while *LLM-FedAvg* only reaches an accuracy of 12%. For the Tiny ImageNet dataset, *LLM-FedAvg* performs the worst, reaching only 0.5% accuracy, which suggests the model is merely guessing randomly rather than learning any meaningful representation from data. However, *LLM-FedAvgNAS* manages to reach an accuracy of 23%, which suggests *LLM-FedAvgNAS* manages to suggest better models and learning rates to achieve good accuracy even for complex datasets, such as Tiny ImageNet. Hence, it can be concluded that while the *LLM-FedAvg* method provides decent results, incorporating NAS using LLM and performing HPO further improves the performance of the provided automated solution.

To further evaluate the effectiveness of *LLM-FedAvgNAS*, experiments are conducted on the CIFAR-10 dataset under a non-IID data distribution setting.To effectively perform Non-IID partitioning, the FedLab framework [50] is utilized, as it provides various partitioning schemes for several datasets. First, *unbalanced* partition scheme is performed where the number of data points per client is kept varied. Here, the number of data points per client is assigned using a log-normal distribution with σ values of 0.2, 0.5, and 0.8 for each experiment. The other scheme used is *hetero dirichlet* scheme, where the number of data points per client remains constant, but the distribution of labels across classes varies for each client. Assigning of the labels per client is done using the Dirichlet distribution with different α values 0.1, 0.5, and 0.8. For both schemes, 100 clients are used with a client fraction of 0.1 to select 10 clients per round. The other parameters for the *LLM-FedAvgNAS* process are the same as the previous. The results presented in Fig. 10 demonstrate that *LLM-FedAvg* performs very poorly under non-iid conditions while *LLM-FedAvgNAS* performs significantly better. However, some decline in accuracy can be observed when the severity of non-iid conditions is increased.

Although *LLM-FedAvgNAS* offers better performance, it comes with some computational overhead during the offline search process before starting the FL task as illustrated in Table V. The table summarizes CPU time, GPU memory usage, and RAM usage averaged over three search rounds for

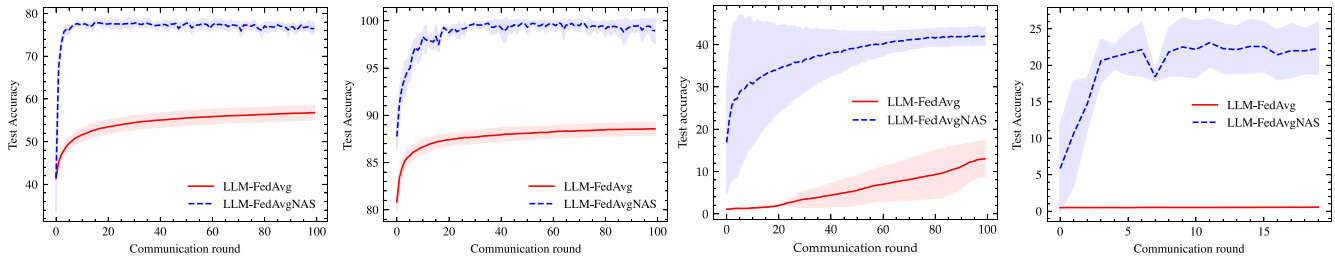

Fig. 9. Results comparison between LLM-FedAvg and LLM-FedAvgNAS for CIFAR-10, Fashion-MNIST, and SVHN datasets. (a) CIFAR-10 dataset. (b) Fashion-MNIST dataset. (c) CIFAR-100 dataset. (d) Tiny ImageNet dataset.

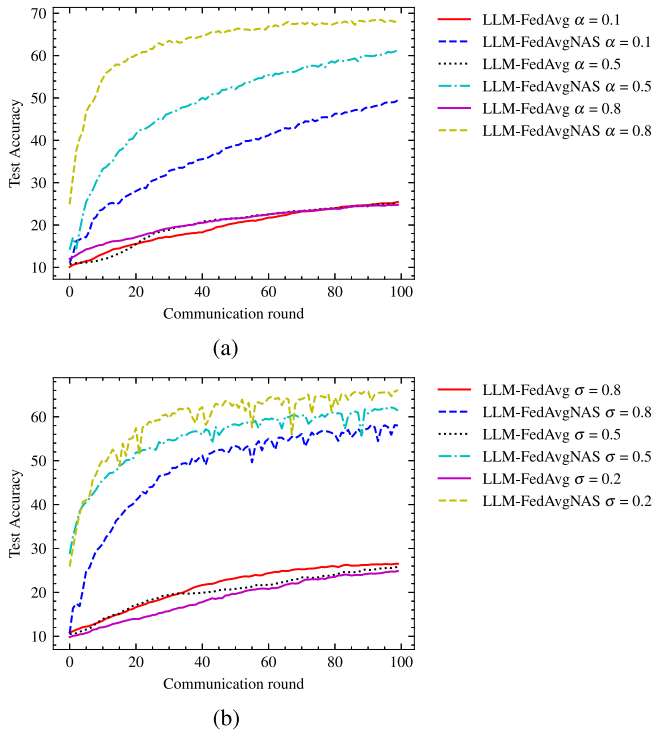

Fig. 10. Results comparison between LLM-FedAvg and LLM-FedAvgNAS for CIFAR-10 with non-iid data distribution. (a) Hetero Dirichlet scheme. (b) Unbalanced scheme.

each dataset. The same parameters used in *LLM-FedAvgNAS* for each dataset for previous experiments are used here as well.

As seen in Table V, resource consumption rises with the increasing complexity of the dataset. Additionally, according to Algorithm 1, the parameters affecting the computational overhead of a single client for a single search round are the number of HPO rounds H, the number of epochs E, the number of hyperparameters Y, and the initial batch size B as these parameters directly affect the running time of the algorithm. However, the resource consumption is not directly affected by the size of the dataset as the search process is not carried out through the entire dataset, but a subset of B batches will be selected per each HPO round which is doubled per each round. To mitigate computational overhead, fine-tuning parameters which affect computational overhead can be an effective strategy.

TABLE V
AVERAGE RESOURCE CONSUMPTION FOR SINGLE SEARCH ROUND IN *LLM-FedAvgNAS* FOR DIFFERENT DATASETS

Dataset	CPU Time (s)	GPU Usage (MB)	Memory Usage (MB)
Fashion-MNIST	167.54	315.0	362.06
CIFAR-10	176.63	336.0	397.07
CIFAR-100	185.36	431.52	493.77
Tiny ImageNet	401.06	1224.0	711.86

VI. CONCLUSION

In this work, a web-based solution for orchestrating FL tasks is introduced, designed to be accessible to researchers without extensive programming knowledge. Complex communications for seamless support for FL tasks are implemented using an efficient WebSocket architecture. Moreover, the integration of intent-based automation, achieved through fine-tuning an LLM, further enhances user accessibility by allowing FL processes to be automated via natural language prompts. Experimental results demonstrate that our automated solution achieves comparable or superior performance to standard methods while maintaining the communication overhead up to 64% lower and CPU time up to 46% lower for the considered scenarios. Furthermore, the integration of NAS and HPO into our solution led to substantial performance gains. This enhancement yielded a 10%–20% improvement in test accuracy across evaluated test cases, successfully mitigating limitations of the automated approach, despite introducing additional computational costs during the search phase.

APPENDIXES

A. Web Form Fields

The input parameters by the end user to the web form and their description are shown in Table VI.

B. JSON File Content

The LLM should produce a compatible JSON output when a user needs to run an FL task through an intent-based method. The associated JSON file for this task is given as follows.

```
{"algo": "Classification",
"minibatch": "16",
"epoch": "5",
"lr": "0.0001",
```

TABLE VI
FIELDS USED IN THE WEB FORM AND THEIR DESCRIPTION

Field Name	Description
Task Name (*text*)	Unique identifier for the task
Host IP (*text*)	IP address of the PS
Client list (*text*)	List of client IP addresses
Algorithm type (*dropdown*)	Classification Regression
Local minibatch size (*int*)	Minibatch size for local training
Local epochs number (*int*)	Number of local training iterations
Learning rate (*decimal*)	Learning rate used for training
Scheduler Type (*dropdown*)	Random Round robin Latency proportional Full
Client Fraction (*decimal*)	Fraction of clients participating in a single round of communication
Test Minibatch size (*int*)	Minibatch size for testing
Communication rounds number (*int*)	Total number of rounds between the PS and the clients
Model file (*file*)	Python model architecture file
Optimizer (*dropdown*)	Adam SGD ...
Loss function (*dropdown*)	Cross entropy loss MSE loss ...
Compression scheme (*dropdown*)	Quantize Topk Randomk No compression

```
"scheduler": "full",
"clientFraction": "1",
"minibatchtest": "32",
"comRounds": "10",
"optimizer": "Adam",
"loss": "CrossEntropyLoss",
"compress": "No",
"dataset": "MNIST"}
```

C. Communication Flow of WebSockets

Unlike in hypertext transfer protocol (HTTP) protocol, WebSocket connection is long-lived, where the connection is closed only when one party deliberately closes the connection or the connection times out due to idle data flow. Initially, an HTTP request is sent to the server from the client requesting a connection upgrade to WebSockets. Websockets connection is established upon connection upgrade response from the server where the connection is kept until the connection is closed by either party. The choice of adopting Websockets for our use case is primarily due to

1) *Bi-Directional Data Transfer:* WebSockets enable bi-directional communication between the Server and the Client. Since in an FL setting, model weights must be communicated between the Server and Client and vice versa, this bi-directional communication enables efficient synchronization of model updates.
2) *Real-Time Communication:* WebSockets facilitate real-time communication by maintaining a persistent connection between the client and the server. Unlike traditional HTTP, which requires establishing a new connection for each request, WebSockets leverage an established connection, eliminating the overhead associated with connection setup. This real-time communication is advantageous for FL, where rapid exchange of model updates is crucial for collaborative learning.
3) *Long-Lived Connection:* WebSockets connection is long-lived, meaning the connection will exist until a client deliberately closes the connection or the connection is inactive for a longer period. This allows in our use case where connections between the user interface, FL server, and clients must be kept alive until respective FL tasks are carried out, which could be minutes or hours in duration.

D. System Prompt

The System Prompt is used to explain the LLM's task. It is kept static and supplemented with each intent given to the LLM, both during fine-tuning and inference. The System Prompt used in our work is given as follows.

> **System Prompt:** Your task is to provide a JSON given an instruction by the user for a federated learning task. The output should be in JSON format. Keys for JSON are algo - Classification/Regression, minibatch - size of minibatch (16), epoch- number of epochs(5), lr - learning rate, scheduler - full/random/round_robin/latency_proportional(full), clientFraction- fraction of clients involved in federated learning (1),comRounds- number of communication rounds in federated learning(10), optimizer-pytorch optimizer(Adam), loss(CrossEntropyLoss), compress-No/quantize(No), dtype-img(img),dataset-dataset used for training. Default values for each key are given inside the bracket. Separated by / are possible values for the relevant key. Your task is to create a JSON with the above keys and extract possible values from a given human prompt as values for the JSON. Respond only to the JSON.

E. Evaluated Prompt-Target Pairs From LLM

Below are some sample prompt-target pairs correctly inferred by the fine-tuned LLM. Observing the samples, it can be noted that the fine-tuned LLM follows the conditional inference rules assigned during the fine-tuning process, which were learned using the fine-tuning dataset. The values directly inferred by the LLM from the user prompt are highlighted in orange for clarity.

Prompt: *Create a federated learning task using the MNIST dataset with a client percentage of 70% from total clients.*

Target: `{ "algo": "Classification," "minibatch": "16," "epoch": "5," "lr": "0.0001," "scheduler": "random,"`

"clientFraction": "0.7", "comRounds": "10," "optimizer": "Adam," "loss": "CrossEntropyLoss," "compress": "No," "dataset": "MNIST"}

Prompt: *Develop a federated learning model on the COCO dataset with 7% of clients, utilizing a random scheduling approach and quantized compression. The training process should employ SGD optimization.*

Target:{ "algo": "Classification", "minibatch": "16", "epoch": "5", "lr": "0.001", "scheduler": "random", "clientFraction": "0.07", "comRounds": "10", "optimizer": "SGD", "loss": "CrossEntropyLoss", "compress": "quantize", "compress_bits": 8, "dataset": "COCO"}

Prompt: *Use a sample of four clients from a total of fifty to carry out a federated learning task on the ImageNET dataset over 30 communication rounds. Perform 15 local epochs. Employ a learning rate of 0.0002157 with Adadelta optimizer. Set minibatch size to 45. Utilize the latency proportional scheduling mechanism.*

Target:{"algo": "Classification", "minibatch": "45", "epoch": "15", "lr": "0.0002157", "scheduler": "latency_proportional", "clientFraction": "0.08", "comRounds": "30", "optimizer": "Adadelta", "loss": "CrossEntropyLoss", "compress": "No","dataset": "ImageNET" }

Prompt: *Perform a Federated learning task on the MNIST dataset with a learning rate of 7/1000.*

Target:{"algo": "Classification", "minibatch": "16", "epoch": "5", "lr": "0.007", "scheduler": "full", "clientFraction": "1", "comRounds": "10", "optimizer": "Adam", "loss": "CrossEntropyLoss", "compress": "No", "dataset": "MNIST"}

Prompt:*Implement a federated learning task on the Weather dataset, involving 35% of clients using latency-based scheduling, and utilize a learning rate of 0.000035. Avoid applying weight compression.*

Target:{"algo": "Classification", "minibatch": "16", "epoch": "5", "lr": "0.000035", "scheduler": "latency_proportional", "clientFraction": "0.35", "comRounds": "10", "optimizer": "Adam", "loss": "CrossEntropyLoss", "compress": "No", "dataset": "Weather"}

ACKNOWLEDGMENT

Views and opinions expressed are, however, those of the author(s) only and do not necessarily reflect those of the European Union. Neither the European Union nor the granting authority can be held responsible for them.

REFERENCES

[1] J. Lin, W. Yu, N. Zhang, X. Yang, H. Zhang, and W. Zhao, "A survey on Internet of Things: Architecture, enabling technologies, security and privacy, and applications," *IEEE Internet Things J.*, vol. 4, no. 5, pp. 1125–1142, Oct. 2017.

[2] H. Habibzadeh, K. Dinesh, O. Rajabi Shishvan, A. Boggio-Dandry, G. Sharma, and T. Soyata, "A survey of healthcare Internet of Things (HIoT): A clinical perspective," *IEEE Internet Things J.*, vol. 7, no. 1, pp. 53–71, Jan. 2020.

[3] E. H. Houssein, M. A. Othman, W. M. Mohamed, and M. Younan, "Internet of Things in smart cities: Comprehensive review, open issues, and challenges," *IEEE Internet Things J.*, vol. 11, no. 21, pp. 34941–34952, Nov. 2024.

[4] I. Ud Din, A. Almogren, and J. J. P. C. Rodrigues, "AIoT integration in autonomous vehicles: Enhancing road cooperation and traffic management," *IEEE Internet Things J.*, vol. 11, no. 22, pp. 35942–35949, Nov. 2024.

[5] B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. A. Y. Arcas, "Communication-efficient learning of deep networks from decentralized data," in *Proc. 20th Int. Conf. Artif. Intell. Statist.*, 2017, pp. 1273–1282.

[6] Y. Xu et al., "A collaborative online AI engine for CT-based COVID-19 diagnosis," 2020, medRxiv, Preprint. [Online]. Available: https://www.medrxiv.org/content/early/2020/05/19/2020.05.10.20096073

[7] K. Tan, D. Bremner, J. L. Kernec, and M. Imran, "Federated machine learning in vehicular networks: A summary of recent applications," in *Proc. Int. Conf. U.K.-China Emerg. Technol. (UCET)*, 2020, pp. 1–4.

[8] S. Niknam, H. S. Dhillon, and J. H. Reed, "Federated learning for wireless communications: Motivation, opportunities, and challenges," *IEEE Commun. Mag.*, vol. 58, no. 6, pp. 46–51, Jun. 2020.

[9] M. M. Amiri and D. Gündüz, "Federated learning over wireless fading channels," *IEEE Trans. Wireless Commun.*, vol. 19, no. 5, pp. 3546–3557, May 2020.

[10] A. Elgabli, J. Park, C. B. Issaid, and M. Bennis, "Harnessing wireless channels for scalable and privacy-preserving federated learning," *IEEE Trans. Commun.*, vol. 69, no. 8, pp. 5194–5208, Aug. 2021.

[11] D. C. Nguyen et al., "Federated learning for smart healthcare: A survey," *ACM Comput. Surv.*, vol. 55, no. 3, pp. 1–37, Feb. 2022. [Online]. Available: https://doi.org/10.1145/3501296

[12] A. M. Elbir, B. Soner, S. Çöleri, D. Gündüz, and M. Bennis, "Federated learning in vehicular networks," in *Proc. IEEE Int. Mediterr. Conf. Commun. Netw. (MeditCom)*, 2022, pp. 72–77.

[13] T. Hiessl, D. Schall, J. Kemnitz, and S. Schulte, "Industrial federated learning–requirements and system design," in *Proc. Int. Conf. Pract. Appl. Agents Multi-Agent Syst.*, 2020, pp. 42–53.

[14] U. M. Aïvodji, S. Gambs, and A. Martin, "IOTFLA: A secured and privacy-preserving smart home architecture implementing federated learning," in *Proc. IEEE Security Privacy Workshops (SPW)*, 2019, pp. 175–180.

[15] D. J. Beutel et al., "Flower: A friendly federated learning research framework," 2020, *arXiv:2007.14390*.

[16] T. Ryffel et al., "A generic framework for privacy preserving deep learning," 2018, *arXiv:1811.04017*.

[17] "Tensorflow federated." 2019, TensorFlow. [Online]. Available: https://www.tensorflow.org/federated

[18] B. Jacob et al., "Quantization and training of neural networks for efficient integer-arithmetic-only inference," in *Proc. IEEE Conf. Comput. Vis. pattern Recognit.*, 2018, pp. 2704–2713.

[19] C. B. Issaid, A. Elgabli, J. Park, M. Bennis, and M. Debbah, "Communication efficient decentralized learning over bipartite graphs," *IEEE Trans. Wireless Commun.*, vol. 21, no. 6, pp. 4150–4167, Jun. 2021.

[20] N. F. Eghlidi and M. Jaggi, "Sparse communication for training deep networks," 2020, *arXiv:2009.09271*.

[21] K. Cobbe et al., "Training verifiers to solve math word problems," 2021, *arXiv:2110.14168*.

[22] Y. Qin et al., "TooLLLM: Facilitating large language models to master 16000+ real-world APIs," 2023, *arXiv:2307.16789*.

[23] S. Thakur et al., "VeriGen: A large language model for verilog code generation," *ACM Trans. Des. Autom. Electron. Syst.*, vol. 29, no. 3, pp. 1–31, Feb. 2024. [Online]. Available: https://doi.org/10.1145/3643681

[24] Q. Gu, "LLM-based code generation method for Golang compiler testing," in *Proc. 31st ACM Joint Eur. Softw. Eng. Conf. Symp. Found. Softw. Eng.* 2023, pp. 2201–2203. [Online]. Available: https://doi.org/10.1145/3611643.3617850

[25] J. Lin, K. Dzeparoska, A. Tizghadam, and A. Leon-Garcia, "AppleSeed: Intent-based multi-domain infrastructure management via few-shot learning," in *Proc. IEEE 9th Int. Conf. Netw. Softw. (NetSoft)*, 2023, pp. 539–544.
[26] S. Kwon, S. Lee, T. Kim, D. Ryu, and J. Baik, "Exploring LLM-based automated repairing of Ansible script in edge-cloud infrastructures," *J. Web Eng.*, vol. 22, no. 6, pp. 889–912, Dec. 2023. [Online]. Available: https://journals.riverpublishers.com/index.php/JWE/article/view/24673
[27] H. Cui, Y. Du, Q. Yang, Y. Shao, and S. C. Liew, "LLMind: Orchestrating ai and IoT with LLM for complex task execution," *IEEE Commun. Mag.*, early access, Sep. 27, 2024, doi: 10.1109/MCOM.002.2400106.
[28] M. Zong, A. Hekmati, M. Guastalla, Y. Li, and B. Krishnamachari, "Integrating large language models with Internet of Things: Applications," *Discov. Internet Things*, vol. 5, no. 1, p. 2, 2025.
[29] T. An, Y. Zhou, H. Zou, and J. Yang, "IoT-LLM: Enhancing real-world IoT task reasoning with large language models," 2024, *arXiv:2410.02429*.
[30] P. Ren et al., "A comprehensive survey of neural architecture search: Challenges and solutions," *ACM Comput. Surv.*, vol. 54, no. 4, pp. 1–34, May 2021. [Online]. Available: https://doi.org/10.1145/3447582
[31] M. Feurer and F. Hutter, "Hyperparameter optimization," in *Automated Machine Learning: Methods, Systems, Challenges*. Cham, Switzerland: Springer, 2019, pp. 3–33.
[32] L. Floridi and M. Chiriatti, "GPT-3: Its nature, scope, limits, and consequences," *Minds Mach.*, vol. 30, pp. 681–694, Dec. 2020. [Online]. Available: https://api.semanticscholar.org/CorpusID:228954221
[33] J. Achiam et al., "GPT-4 technical report," 2023, *arXiv:2303.08774*.
[34] H. Touvron et al., "LLaMA: Open and efficient foundation language models," 2023, *arXiv:2302.13971*.
[35] H. Touvron et al., "LLaMA 2: Open foundation and fine-tuned chat models," 2023, *arXiv:2307.09288*.
[36] A. Q. Jiang et al., "Mistral 7B," 2023, *arXiv:2310.06825*.
[37] A. Q. Jiang et al., "Mixtral of experts," 2024, *arXiv:2401.04088*.
[38] A. Vaswani et al., "Attention is all you need," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 30, 2017, pp. 1–15.
[39] E. J. Hu et al., "LoRa: Low-rank adaptation of large language models," 2021, *arXiv:2106.09685*.
[40] T. Dettmers, A. Pagnoni, A. Holtzman, and L. Zettlemoyer, "QLoRa: Efficient finetuning of quantized LLMs," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 36, 2024, pp. 1–26.
[41] "React framework." React Developers. 2024. [Online]. Available: https://react.dev/
[42] "Redux framework." Redux Developers. 2024. [Online]. Available: https://redux.js.org/
[43] T. Brown et al., "Language models are few-shot learners," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 33, 2020, pp. 1877–1901.
[44] M. Escarda-Fernández et al. "LLMs on the fly: Text-to-JSON for custom API calling." Jul. 2024. [Online]. Available: https://www.scribd.com/document/808346931/LLMs-on-the-Fly-Text-To-JSON-for-Custom-API-Calling
[45] L. Huang et al., "A survey on hallucination in large language models: Principles, taxonomy, challenges, and open questions," *ACM Trans. Inf. Syst.*, vol. 43, no. 2, pp. 1–55, 2023.
[46] A. Betlen. "LLaMA.cpp binding for Python." 2024. [Online]. Available: https://github.com/abetlen/llama-cpp-python
[47] G. Gerganov. "LlaMA.cpp library." 2024. [Online]. Available: https://github.com/ggerganov/llama.cpp
[48] I. Loshchilov and F. Hutter, "Decoupled weight decay regularization," 2017, *arXiv:1711.05101*.
[49] D. Han. "Unsloth." 2023. [Online]. Available: https://github.com/unslothai/unsloth.git
[50] D. Zeng, S. Liang, X. Hu, H. Wang, and Z. Xu, "FedLab: A flexible federated learning framework," *J. Mach. Learn. Res.*, vol. 24, no. 100, pp. 1–7, 2023.

Chamith Mawela received the B.Sc. (Hons.) degree in electrical and information engineering from the University of Ruhuna, Matara, Sri Lanka, in 2019, and the M.Sc. degree in computer science and engineering from University of Oulu, Oulu, Finland, in 2024.

He is a Project Researcher with the Center of Wireless Communications, University of Oulu. Before that he worked as a Graduate Research Assistant at the Center of Wireless Communications, University of Oulu. He also has nearly four years of experience in the software industry in Sri Lanka. His research interests focus on federated learning and applications of large language models, and reinforcement learning for efficient communication.

Chaouki Ben Issaid (Member, IEEE) received the Diplôme d'Ingénieur from l'École Polytechnique de Tunisie, La Marsa, Tunisia, in 2013, the M.Sc. degree in applied mathematics and computational science, and the Ph.D. degree in statistics from King Abdullah University of Science and Technology, Thuwal, Saudi Arabia, in 2015 and 2019, respectively.

He is a Senior Researcher and an Adjunct Professor (docent) with the University of Oulu, Oulu, Finland. Before that, he was a Postdoctoral Researcher with the Centre of Wireless Communications, University of Oulu from 2020 to 2023. His current research interests include communication-efficient distributed machine learning with applications to wireless communication.

Mehdi Bennis (Fellow, IEEE) is a Professor with the Centre of Wireless Communications, University of Oulu, Oulu, Finland, Academy of Finland Research Fellow and Head of the Intelligent Connectivity and Networks/Systems Group. He has published more than 200 research papers in international conferences, journals and book chapters. His main research interests are in radio resource management, heterogeneous networks, game theory and distributed machine learning in 5G networks, and beyond.

Dr. Bennis has been the recipient of several prestigious awards, including the 2015 Fred W. Ellersick Prize from the IEEE Communications Society, the 2016 Best Tutorial Prize from the IEEE Communications Society, the 2017 EURASIP Best paper Award for the Journal of Wireless Communications and Networks, the all-University of Oulu award for research, the 2019 IEEE ComSoc Radio Communications Committee Early Achievement Award, and the 2020 Clarviate Highly Cited Researcher by the Web of Science. He is an Editor of IEEE TRANSACTIONS ON COMMUNICATIONS and the Specialty Chief Editor of *Data Science for Communications* (Frontiers) in *Communications and Networks* journal.